



UNIVERSITÀ
DEGLI STUDI
DI PALERMO



Il World Wide Web e il protocollo HTTP

CORSO DI PROGRAMMAZIONE WEB E MOBILE

A.A. 2025/2026

ING. LUCA CRUCIATA

Sommario

- Generalità sul web
- Iper testi e architettura dell'informazione
- Concetto di URL e funzionamento del servizio web
- Il protocollo HTTP

Il World Wide Web

- Il **World Wide Web** (WWW) è un servizio di consultazione di documenti organizzati secondo una struttura ipertestuale distribuita sulla rete Internet.
- Il WWW è sicuramente il componente più visibile di Internet, tanto che si tende ad identificarlo con quest'ultima.

Evitiamo la confusione...

È importante fare una distinzione fra cosa si intende per INTERNET e cosa per WWW.

- **Internet** è l'infrastruttura, ovvero la rete fisica che collega computer, server e dispositivi di tutto il mondo. È come un'autostrada globale che permette ai dati di viaggiare.
- **World Wide Web (WWW)** è uno dei tanti servizi che viaggiano su Internet. È composto da siti web, pagine, link e contenuti che possiamo navigare con un browser .

Se vogliamo fare un analogia internet è un'autostrada, mentre WWW sono le macchine e i cartelli che ti portano dove vuoi.

Il World Wide Web

- Si possono distinguere due grandi usi della tecnologia web:
 - Come strumento ipertestuale di comunicazione (in forma nativa)
 - Come strumento di comodo interfacciamento di altri servizi di rete (accesso a banche dati, e-commerce, sistemi informativi aziendali, etc.)
- Nella seconda forma la tecnologia del web viene usata anche in ambienti “chiusi”, come le reti aziendali, cioè non direttamente collegati ad internet. In questa accezione nasce il concetto di [Intranet](#).

Cos'è un sito web

- Un sito web può essere definito come un particolare tipo di *ipertesto*.
- **Ipertesto**: organizzazione di materiale informativo (in genere multimediale) in forma tale da consentirne una fruizione non lineare in cui sono presenti dei percorsi di fruizione predefiniti dal progettista secondo un preciso progetto comunicativo.
- Definizione meno formale: «*Un ipertesto, come un qualunque sito web, è invece costituito da un insieme di elementi, pagine di testo od oggetti multimediali che possono essere consultati nell'ordine scelto dallo stesso lettore*»

Esempio di ipertesto

WIKIPEDIA

L'enciclopedia libera

Cerca in Wikipedia

Ricerca

Fai una donazione Registrati Entra ...

Wiki Loves FOLKLORE

Photograph your local culture, help Wikipedia and win!

Informatica

🌐 162 lingue ▼

Indice nascondi

Inizio

- Etimologia e significato
- Storia
- > Caratteristiche
- > Aree tematiche
 - Insegnamento, titoli di studio e certificazioni
- Note
- Bibliografia
- Voci correlate
- Altri progetti
- Collegamenti esterni

Voce Discussione

Leggi Modifica Modifica wikitesto Cronologia Strumenti ▼

Disambiguazione – Se stai cercando l'*azienda statunitense*, vedi **Informatica (azienda)**.

L'**informatica** è la disciplina scientifica che si occupa dello studio dell'elaborazione delle informazioni e delle sue applicazioni pratiche. Più nel dettaglio, essa si concentra sulla rappresentazione, organizzazione e gestione automatica delle informazioni. Il termine ha origine dal francese *informatique*, unione delle parole *information* (informazione) e *automatique* (automatica), e fu introdotto dal fisico francese **Philippe Dreyfus** nel 1962. Pur essendo una scienza autonoma, l'informatica è strettamente legata all'uso del calcolatore, che rappresenta uno dei principali strumenti per il suo sviluppo; l'evoluzione dell'informatica è infatti andata di pari passo con i progressi tecnologici dei computer.^[1]

Una rappresentazione artistica della macchina di Turing

Si accompagna e si integra o è di supporto a tutte le discipline scientifiche e, come **tecnologia**, pervade pressoché qualunque "mezzo" o "strumento" di utilizzo comune e quotidiano, tanto che (quasi) tutti siamo in qualche modo utenti di servizi informatici. La valenza dell'informatica in termini socioeconomici ha scalato in pochi anni la **piramide di Anthony**, passando da operativa (in sostituzione o a supporto di compiti semplici e ripetitivi), a tattica (a supporto della pianificazione o gestione di breve termine), a strategica. In tale ambito, l'informatica è diventata talmente strategica nello **sviluppo economico** e sociale delle popolazioni che il non poterla sfruttare, uno status ribattezzato con l'espressione **divario digitale**, è un problema di interesse planetario.

Assieme all'**elettronica** e alle **telecomunicazioni**, unificate insieme sotto la denominazione **tecnologie dell'informazione e della comunicazione** (TIC), rappresenta quella disciplina e allo stesso tempo quel **settore economico** che ha dato vita e sviluppo alla **terza rivoluzione industriale**, attraverso quella che è comunemente nota come **rivoluzione digitale**.

Aspetto nascondi

Testo

☐ Piccolo
 ☒ Standard
 ☐ Grande

Larghezza

☒ Standard
 ☐ Largo

Colore (beta)

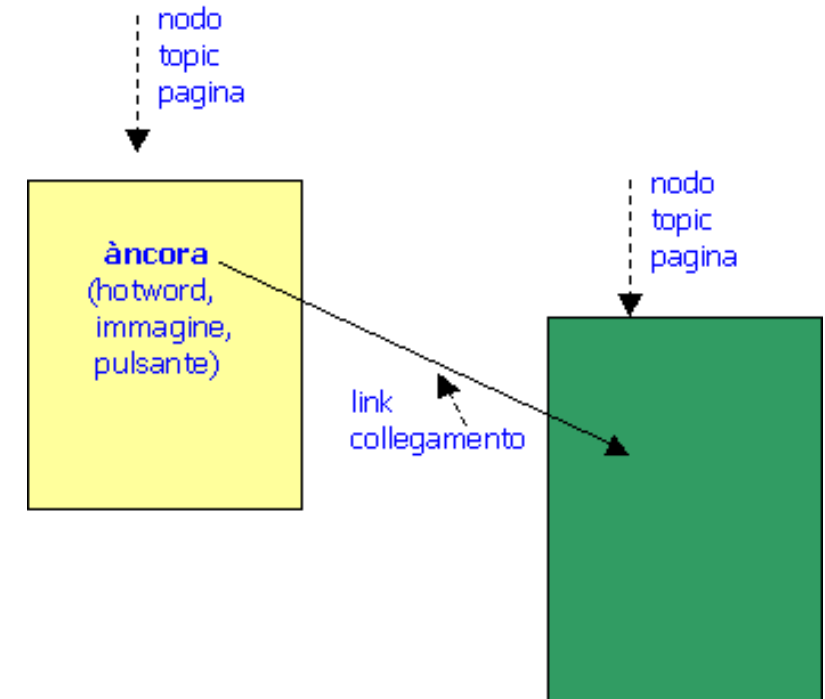
☐ Automatico
 ☒ Chiaro
 ☐ Scuro

Elementi di un ipertesto

- In un qualunque ipertesto (e quindi anche nel web) si possono individuare i seguenti elementi fondamentali:
- **Nodo**: unità informativa autosufficiente, testuale e non; in genere in un nodo sono presenti più elementi eterogenei (testo, grafica, filmati etc.)
- **Ancora**: punto di origine o di termine di un collegamento ad un'altra unità.
- **Link**: collegamento ad un'altra unità; può avere diverse funzioni (salto, riferimento, comando, nota esplicativa etc.) ed è agganciato a due ancore.

Esempio di nodo, ancora e link

- Una pagina di Wikipedia è un **nodo**.
- In una pagina web, una parola evidenziata in blu con un link è un'**ancora**. (L'ancora è un punto specifico dentro un nodo a cui si può collegare un link.)
- Un **link** su un sito web che ti porta a un'altra pagina. (Si può collegare a un altro nodo o a un'altra parte dello stesso nodo.)

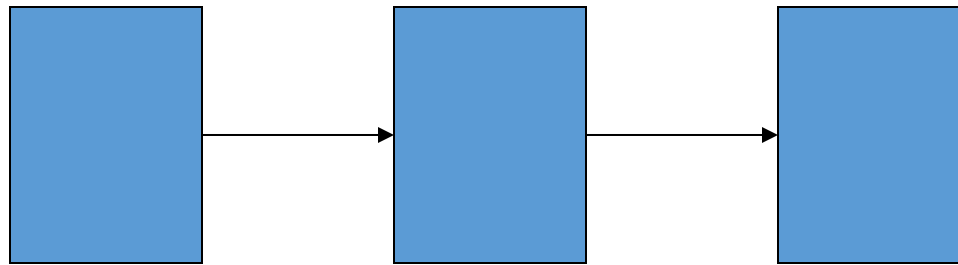


Architettura dell'Informazione

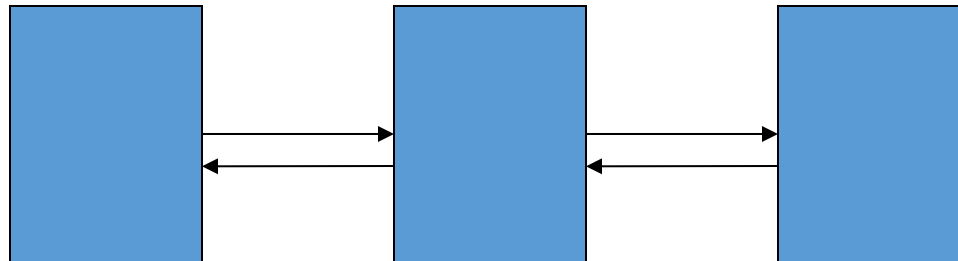
- L'Architettura dell'informazione di un sito web consiste nel suo **design strutturale** atto a facilitare l'esecuzione dei compiti e l'accesso intuitivo ai contenuti.
- Si può dire che l'Architettura dell'informazione sia la combinazione di **organizzazione, categorizzazione e schemi di navigazione** dei contenuti all'interno di un sito.

Architettura dell'informazione

Le informazioni si seguono in sequenza, utile per guide o corsi online.



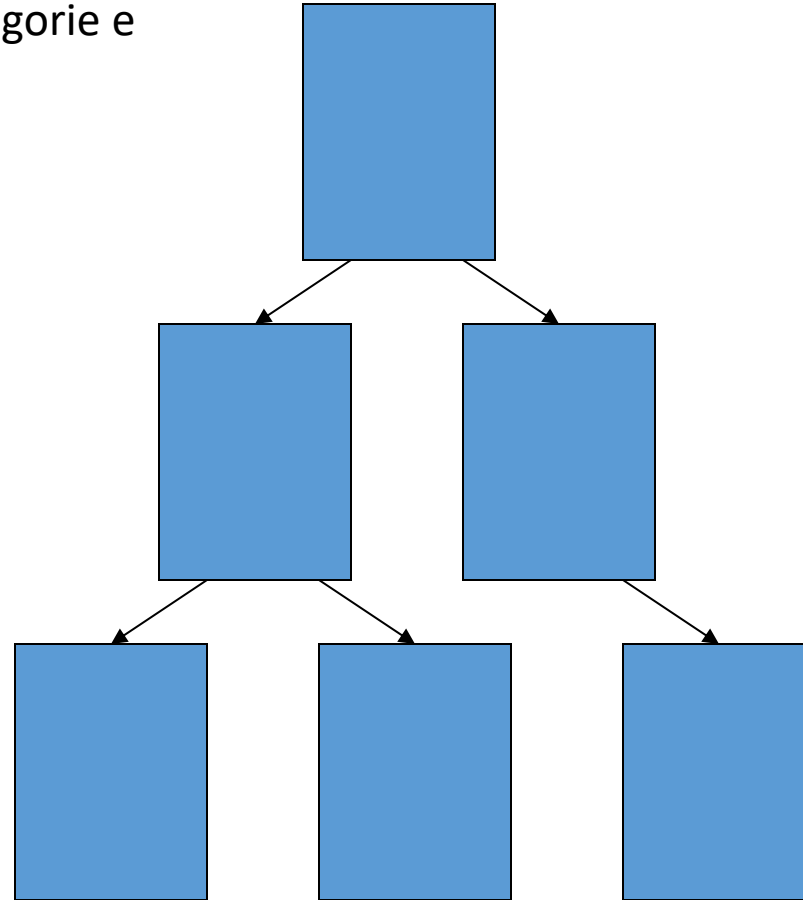
Lista



Lista doppia

Architettura dell'informazione

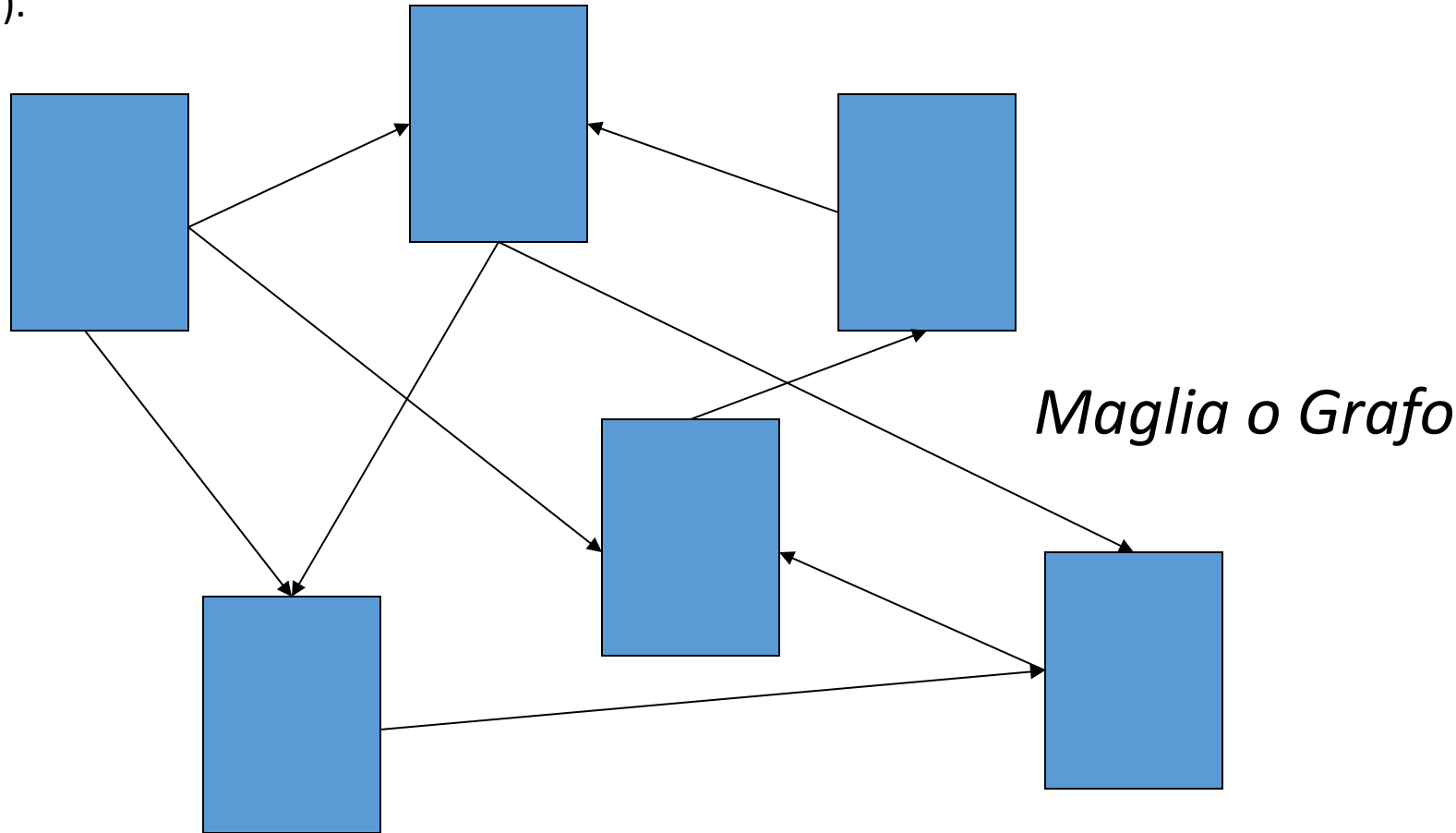
La più comune, con una home page principale e sottopagine organizzate in categorie e sottocategorie.



Albero

Architettura dell'informazione

Tutte le pagine sono collegate tra loro in modo non gerarchico
(es. Wikipedia).



Risorse informative: URI, URL, URN

- **URI: *Uniform Resource Identifier***, è una stringa che identifica univocamente una risorsa generica sulla rete. (Come il CF identifica una persona)
- Gli URI rendono disponibili le risorse secondo una varietà di protocolli quali HTTP, FTP, ecc.

Risorse informative: URI, URL, URN

- **URN: Uniform Resource Name**, è un URI che identifica una risorsa mediante un "nome" in un particolare dominio di nomi ("namespace") senza nessuna indicazione sulla sua esistenza e/o posizione nella rete.
 - Sintassi **urn**:<*Namespace Identifier*>:<*Namespace Specific String*>
 - Es. **urn:isbn:0-395-36341-1** (Identifica *quel* libro specifico tramite isbn, ma non ti dice se si trova nella biblioteca sotto casa o su Amazon.)

Risorse informative: URI, URL, URN

- **URL: Uniform Resource Locator** è un URI che, oltre a identificare una risorsa, fornisce mezzi per agire su o per ottenere una rappresentazione della risorsa descrivendo il suo meccanismo di accesso primario o la sua "ubicazione" ("location") in una rete.
- *Ogni singolo elemento di una pagina web è da considerarsi risorsa*

Nella pratica dello sviluppo web moderno, usiamo quasi sempre gli **URL** perché abbiamo bisogno che il browser sappia esattamente dove andare per ottenere i dati (HTML, immagini, API)

URI, URN, URL

- **URI** (Uniform Resource Identifier) → Identifica in modo univoco una risorsa, ma senza specificare come accedervi.
- **URN** (Uniform Resource Name) → È un tipo di URI che serve solo a identificare una risorsa in modo permanente, senza indicare la sua posizione.
- **URL** (Uniform Resource Locator) → È un tipo di URI che specifica dove si trova la risorsa e come accedervi (di solito con un protocollo come HTTP, FTP, ecc.).

URL

- Una URL ha una sintassi generica del tipo:
<protocollo><indirizzo><percorso>
- <http://www.unipa.it/persone/docenti/p/roberto.pirrone>
- <mailto:luca.cruciata@unipa.it>
- <ftp://ftp.unina.it/pub/linux/redhat/>

Tipi di URL

- Richiesta di un documento ipertestuale:

`http://<nome_server>[:<numero_porta>]/<percorso>[/<nome_file_html>][#<ancora>]`

- Numero porta: la porta TCP di erogazione del servizio, se diversa da 80 (default).
- Se il nome del file html manca, si assume index.html, default.html o index.php, altrimenti bisogna specificarlo a priori.
- Ancora: riferimento ad una zona precisa all'interno della pagina (usata per i riferimenti incrociati interni).

Tipi di URL

- Richiesta di trasferimento file sicuro:

`sftp://[<username>[:<password>]@]<nome server>/ <percorso>/<nome file>`

- *Username e password*: identificazione di un utente specifico, altrimenti si assume l'utente *anonymous* senza password.

Tipi di URL

- Richiesta di terminale remoto sicuro:

`ssh://<utente>[:<password>] @<nome server>[:<numero porta>]`

- Richiesta di invio posta elettronica:

`mailto:<nome utente>@<nome server>`

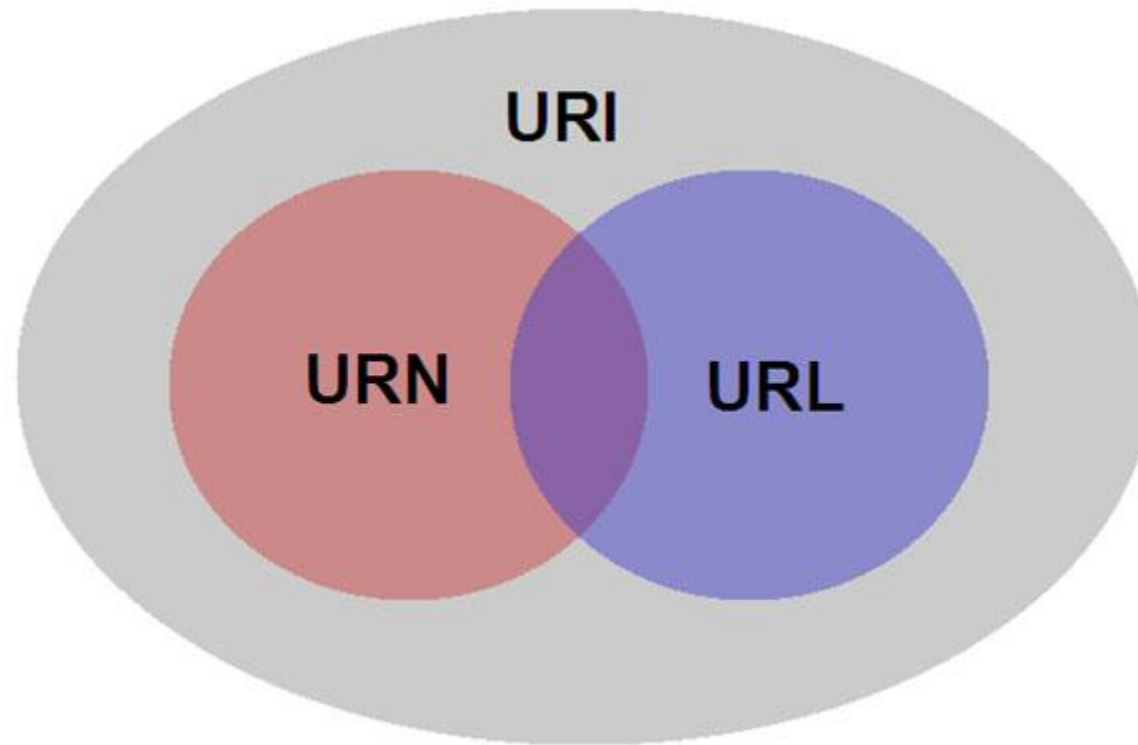
- Spesso il nome del server coincide con quello del dominio: è il DNS del dominio a conoscere l' esatto indirizzo del server)

Esempio pratico

Immagina di essere in una biblioteca e di voler trovare un libro.

- **URN** (Nome Unico del Libro) : Ogni libro ha un codice ISBN (es. ISBN:978-3-16-148410-0). Questo codice identifica solo quel libro, indipendentemente da dove si trovi. Esempio di URN: urn:isbn:9783161484100
- **URL** (Dove trovare il libro): Se vuoi leggere il libro online, hai bisogno di un link specifico, come <https://www.miabiblioteca.it/libro123> Questo ti dice dove è disponibile e come accedervi (tramite HTTPS). Esempio di URL: <https://it.wikipedia.org/wiki/URI>
- **URI** (Il contenitore generale): Sia il codice ISBN (URN) che il link al sito (URL) sono tipi di URI.

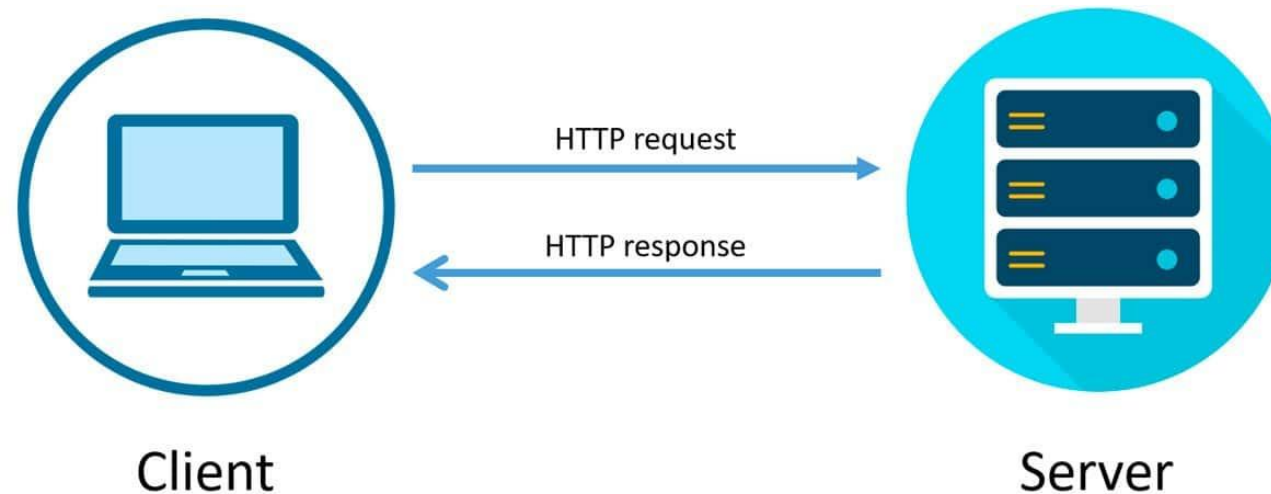
Riassunto URI, URN, URL



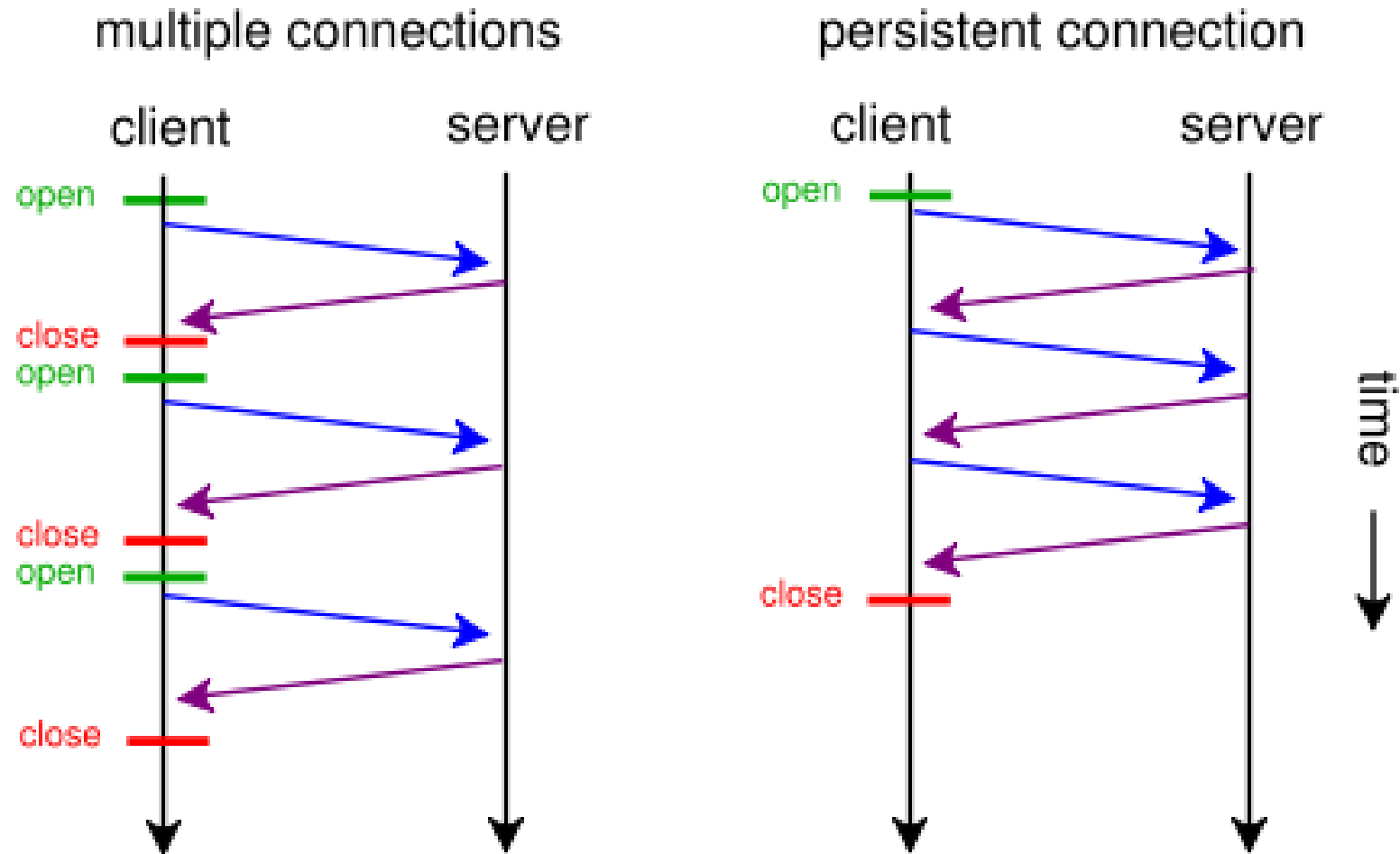
<https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.geeksforgeeks.org%2Fdifference-between-url-uri-and-urn-in-java%2F&psig=AOvVaw0sc0D6yi3YGBef7NFPgqk7&ust=1741079523656000&source=images&cd=vfe&opi=89978449&ved=0CBcQjhqxqFwoTCLDdm-fl7YsDFQAAAAAdAAAAABAE>

HTTP (Hyper-Text Transfer Protocol)

- HTTP: Protocollo di applicazione – su TCP
- Modello di comunicazione client-server
- La comunicazione avviene attraverso coppie richiesta – risposta su stream TCP (HTTPRequest, HTTPResponse)



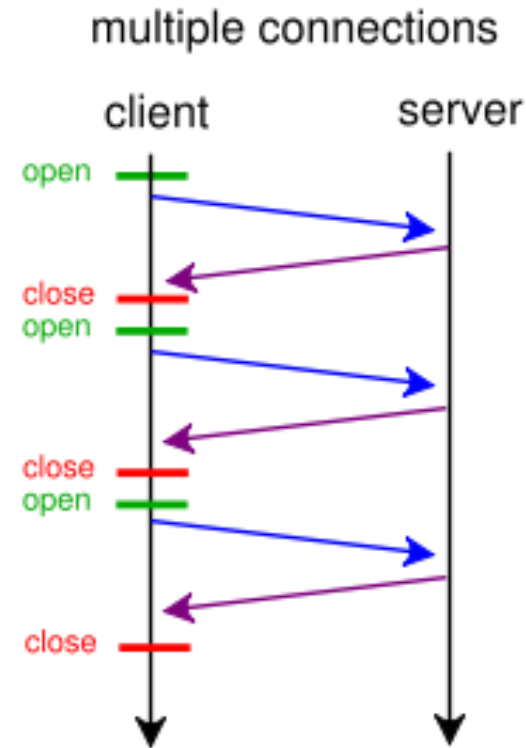
Modello di connessione in HTTP



Connessioni Multiple o Non persistenti

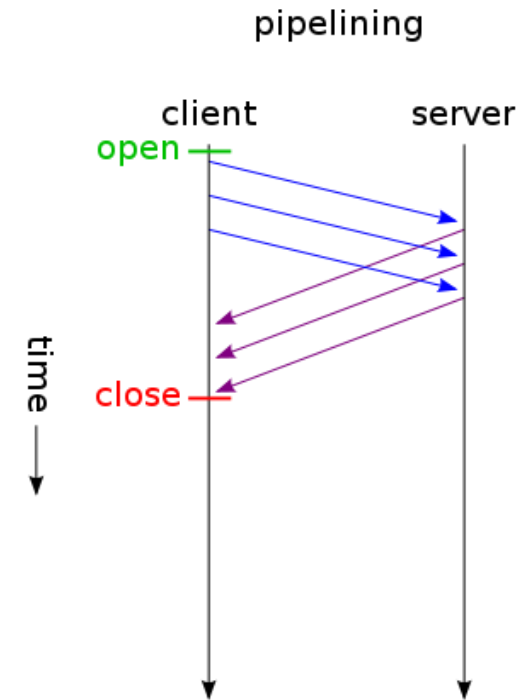
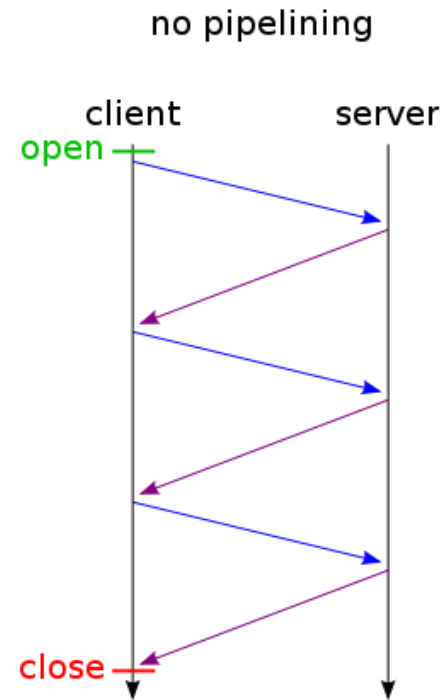
Come si vede dall'immagine, per ogni coppia di richiesta e risposta la connessione viene chiusa.

Quindi per una nuova richiesta, verrà stabilita una nuova connessione.



Connessioni persistenti

- Coppie richiesta – risposta multipla sulla stessa connessione TCP
- Pipelining
 - Più richieste partono di seguito prima che vengano ricevute le risposte
 - Tipicamente per le richieste GET



HTTP Request

```
GET /search?q=Introduction+to+XML+and+Web+Technologies HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.7.2)
↳ Gecko/20040803
Accept: text/xml,application/xml,application/xhtml+xml,
↳ text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5
Accept-Language: da,en-us;q=0.8,en;q=0.5,sw;q=0.3
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Referer: http://www.google.com/
```

Campi della risposta HTTP

Metodi GET e POST

HOST: Che URL stiamo richiedendo?

User-agent: browser/S.O./piattaforma software

Accept-...: Formati dei dati, compressione, linguaggio e set dei caratteri accettati dal browser

Keep-alive, Connection: Tempo di mantenimento della connessione che viene impostata come “persistente” per quel lasso di tempo

HTTP Response

```
HTTP/1.1 200 OK
Date: Fri, 17 Sep 2009 07:59:01 GMT
Server: Apache/2.0.50 (Unix) mod_perl/1.99_10 Perl/v5.8.4
↳ mod_ssl/2.0.50 OpenSSL/0.9.7d DAV/2 PHP/4.3.8 mod_bigwig/2.1-3
Last-Modified: Tue, 24 Feb 2009 08:32:26 GMT
ETag: "ec002-afa-fd67ba80"
Accept-Ranges: bytes
Content-Length: 2810
Content-Type: text/html

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<html>...</html>
```

Linea di stato

Il codice di stato che informa il client sul soddisfacimento o meno della richiesta e sul motivo

Codici di stato HTTP

- 1xx Informational
 - Richiesta ricevuta, continua l'elaborazione
 - **100** Continue
 - Il server ha ricevuto l'header della richiesta e il client deve procedere ad inviare il corpo della richiesta (richieste di tipo POST)
- 2xx Success
 - **200** OK
 - Risposta standard per le richieste HTTP andate a buon fine

Codici di stato HTTP

- 3xx Redirection
 - Il client deve eseguire ulteriori azioni per soddisfare la richiesta
- 301 Moved Permanently
 - Questa e tutte le future richieste andranno dirette ad un altro URI
- 304 Not Modified
 - Non c'è bisogno di ritrasmettere la risorsa: è una redirectione implicita alla cache

Codici di stato HTTP

- 4xx Client Error
 - La richiesta è sintatticamente scorretta o non può essere soddisfatta
- 400 Bad Request
 - La richiesta non può essere soddisfatta a causa di errori di sintassi
- 401 Unauthorized
- 403 Forbidden
 - La richiesta è legittima ma il server si rifiuta di soddisfarla
- 404 Not Found
 - La risorsa richiesta non è stata trovata

Codici di stato HTTP

- 4xx Client Error
 - La richiesta è sintatticamente scorretta o non può essere soddisfatta
- 405 Method Not Allowed
 - La richiesta è stata eseguita usando un metodo non permesso

Codici di stato HTTP

- 5xx Server Error
 - Il server ha fallito nel soddisfare una richiesta apparentemente valida
- 500 Internal Server Error
 - Messaggio di errore generico senza alcun dettaglio
- 503 Service Unavailable
 - Il server non è al momento disponibile. Generalmente è una condizione temporanea

HTTP Response

```
HTTP/1.1 200 OK
Date: Fri, 17 Sep 2009 07:59:01 GMT
Server: Apache/2.0.50 (Unix) mod_perl/1.99_10 Perl/v5.8.4
↳ mod_ssl/2.0.50 OpenSSL/0.9.7d DAV/2 PHP/4.3.8 mod_bigwig/2.1-3
Last-Modified: Tue, 24 Feb 2009 08:32:26 GMT
ETag: "ec002-afa-fd67ba80"
Accept-Ranges: bytes
Content-Length: 2810
Content-Type: text/html

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<html>...</html>
```

Campi della risposta HTTP

Linea di stato

Date La data della risposta

Server Identificazione server/S.O./moduli e librerie installati

Last-modified La data di ultima modifica della risorsa

Etag Entity tag (opzionale) usato come chiave della risorsa per la gestione efficiente della cache del browser

Accept-Ranges Il server esaudirà anche richiesta parziali di una risorsa inviando gruppi di byte

Content-Length Lunghezza in byte

Content-Type tipo MIME della risposta

Cache

- La cache è usata dai browser, dai server e anche sulla rete attraverso i *proxy server*
 - Cache: uno spazio di archiviazione e di memoria centrale, gestito dal software che la usa, per l'accesso efficiente alle risorse già precaricate e indicizzate
- Direttiva Cache-Control:
 - **no-store** non conserva in cache
 - **no-cache** può conservare in cache (richiede validazione)
 - **public** può conservare in cache
 - **private** per utenti singoli
 - **max-age** scadenza
 - **must-revalidate** obbligo di validazione

MIME

- Multipurpose Internet Mail Exchange
 - [RFC2045](#), [RFC2046](#), [RFC2047](#), [RFC4288](#), [RFC4289](#), [RFC2049](#)
 - Usato come standardizzazione dei messaggi di posta elettronica multi-formato, cioè con allegati
 - I messaggi mail sarebbero di puro testo – un carattere è codificato in un byte
 - Le etichette MIME battezzano gli allegati che sono codificati in testo equivalente usando la codifica *Base64*
 - Esteso alla gestione di qualunque contenuto trasferito con HTTP

MIME – Base64

- Il flusso dati da codificare viene suddiviso in sequenze da 6 bit ciascuna
- Ogni intero da 6 bit viene convertito in un carattere secondo la tabella di codifica
- 3 byte → 4 caratteri codificati
 - Es. “Man” → “TWFu”
 - In caso di mancato allineamento si usa “=” per il padding a destra

Base64 Encoding Table

Value	Char	Value	Char	Value	Char	Value	Char
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

MIME – Base64

“Bis”: | 01000010 | 01101001 | 01110011 (binario)
| 01000010 | 01101001 | 01110011 (prendo 6 bit)
“ Q m l z ” (base64)

“La”: | 01001100 | 01100001
| 01001100 | 01100001 | 00000000
“ T G E = ”

Link dove provare la codifica: <https://kryptos.altervista.org/ascii/>

Base64 Encoding Table

Value	Char	Value	Char	Value	Char	Value	Char
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

MIME – Content Type

- Etichette *tipo/sottotipo*
 - text/plain - il formato testo (estensione .txt)
 - text/html – il testo HTML
 - application/octet-stream - il flusso dei byte di un programma eseguibile
 - image/jpeg, audio/mp3, video/mp4 – audio e video
 - application/msword - documento word
 - ...

HTTP Response

```
HTTP/1.1 200 OK
Date: Fri, 17 Sep 2009 07:59:01 GMT
Server: Apache/2.0.50 (Unix) mod_perl/1.99_10 Perl/v5.8.4
↳ mod_ssl/2.0.50 OpenSSL/0.9.7d DAV/2 PHP/4.3.8 mod_bigwig/2.1-3
Last-Modified: Tue, 24 Feb 2009 08:32:26 GMT
ETag: "ec002-afa-fd67ba80"
Accept-Ranges: bytes
Content-Length: 2810
Content-Type: text/html

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<html>...</html>
```

- **Corpo della risposta**
 - Il flusso di caratteri che compongono la risorsa se HTML
 - Il contenuto binario di un'immagine o altro dato multimediale
 - Un file Javascript o CSS esterno
 - ...

I due metodi per inviare dati al server

- Andare su:
https://tryphp.w3schools.com/showphp.php?filename=demo_form_post
 - Attivare gli strumenti per sviluppatori da Firefox e osservare il traffico di Rete
- Andare su:
https://tryphp.w3schools.com/showphp.php?filename=demo_form_get
 - Attivare gli strumenti per sviluppatori da Firefox e osservare il traffico di Rete

GET e POST

- I due *metodi* base di richiesta di risorse
 - GET è la richiesta standard di risorse e informazioni
 - Può essere usata anche per inviare informazioni
 - POST è di norma usata per inviare informazioni al server e innescare ulteriori elaborazioni
 - L'invio di informazioni ad un server HTTP si effettua tramite i *Form* HTML

Form HTML

- Tag HTML dedicati per l'inserimento di dati testuali, selezione da menu, radio button, checkbox ...

```
<h3>The Poll Service</h3>
<form action="http://freewig.brics.dk/users/laudrup/soccer.jsp"
      method="post">
  Who wins the World Cup 2006?
  <select name="bet">
    <option value="br">Brazil!</option>
    <option selected value="dk">Denmark!</option>
    <option value="other country">someone else?</option>
  </select><br>
  Please enter your email address:
  <input type="text" name="email"><br>
  <input type="submit" name="send" value="Go!">
</form>
```

The Poll Service

Who wins the World Cup 2006?

Please enter your email address:

GET o POST?

- GET

- Va bene per **richiedere** dati
- Non sicuro: l'informazione è nella URL (?user=alex&age=25)
- Limiti lunghezza della URL

- POST

- Richiede che i dati nel corpo messaggio siano in qualche modo “subordinati” alla URI richiesta, quindi serve per **inviare** dati
- Non sicuro: l'informazione è nel corpo della HTTP Request – poco più sicuro di GET
- Consente diversi tipi di encoding
- Consente di usare una cache delle richieste

URL encoding

- Nella URL di richiesta possono esserci spazi sia nei nomi sia nei valori dei dati e può essere utilizzata anche una tabella dei caratteri non standardizzata
- La stringa viene “ricodificata” usando il set di caratteri Unicode (UTF8)
 - Gli spazi vengono sostituiti da “+” ovvero dalla sequenza “%20”

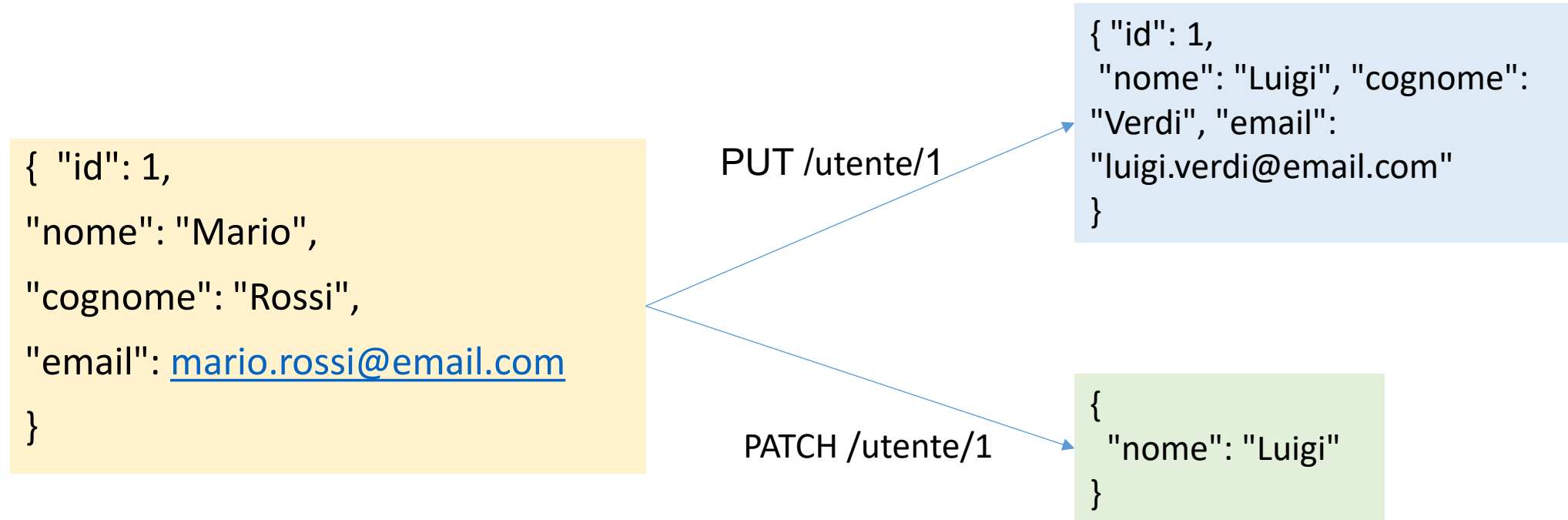
Altri metodi HTTP

- HEAD
 - Identico a GET, ma solo per la richiesta dell'intestazione della risposta; utile per analizzare i metadati
- PUT
 - Crea o modifica l'URI richiesta con il contenuto del messaggio
- DELETE
 - Cancella la risorsa specificata

Altri metodi HTTP

- TRACE
 - Rimanda indietro la risorsa per consentire al client di verificare le modifiche inserite da server intermedi lungo la connessione (come i proxy)
- OPTIONS
 - Restituisce i metodi supportati dal server relativi alla URL specificata nella richiesta
 - OPTIONS * effettua il controllo di funzionalità di un web server
- CONNECT
 - Richiede a un proxy di stabilire un *HTTP tunneling* con l'host di destinazione
- PATCH
 - Applica modifiche parziali alla risorsa richiesta al contrario di PUT che la sostituisce del tutto

Differenza fra PUT e PATCH



Esempio online

<https://jsonplaceholder.typicode.com/>

```
fetch('https://jsonplaceholder.typicode.com/posts', {  
  method: 'POST',  
  body: JSON.stringify({  
    title: 'foo',  
    body: 'bar',  
    userId: 1,  
  }),  
  headers: {  
    'Content-type': 'application/json; charset=UTF-8',  
  },  
})  
.then((response) => response.json())  
.then((json) => console.log(json));
```

Esempio POST

```
fetch('https://jsonplaceholder.typicode.com/posts/1', {  
  method: 'PUT',  
  body: JSON.stringify({  
    id: 1,  
    title: 'foo',  
    body: 'bar',  
    userId: 1,  
  }),  
  headers: {  
    'Content-type': 'application/json; charset=UTF-8',  
  },  
})  
.then((response) => response.json())  
.then((json) => console.log(json));
```

Esempio PUT

Metodi safe, idempotenti e cacheable

Metodi safe

- Sono quelli la cui esecuzione non crea effetti collaterali sul server
- HEAD, GET, OPTIONS e TRACE
- Richiedono informazioni
- POST, PUT, DELETE e PATCH invece modificano anche parzialmente una risorsa

Metodi idempotenti

- Multiple richieste identiche lasciano lo stato del server inalterato
- GET, HEAD, PUT, DELETE, OPTIONS e TRACE
- Esempio: una richiesta DELETE per eliminare una risorsa avrà sempre lo stesso risultato, anche se ripetuta.

Metodi cacheable

- Possono essere gestiti con la web cache per evitare il carico sul server
- GET, POST e HEAD

Autenticazione in HTTP

- Tramite Form che inviano coppie username/password che vengono gestite da applicazioni lato server
 - Soluzione più sicura
- HTTP Basic
- HTTP Digest
 - Sicuro: usa cifratura MD5 di username e password

Autenticazione HTTP Basic

- Il server invia al client che ha richiesto la risorsa un *challenge*:
 - HTTP/1.1 401 Authorization Required WWW-Authenticate: Basic realm="The Doe Family Site"
- Il client invia username e password in un'unica stringa codificata in *Base64*
 - Authorization: Basic emFjaGFyaWFzOmFwcGxlcGllCg==
 - La stringa è la concatenazione di username e password col carattere ":"
 - Autenticazione in chiaro: <http://pippo:pluto@www.somewhere.com>

Autenticazione HTTP Basic

- *realm*:

- È una stringa che definisce il nome di un “protection space” cioè un gruppo di risorse che ricadono nello stesso ambito di autenticazione e sicurezza
- Possono definirsi diversi realm per l’autenticazione a porzioni diverse di risorse dello stesso server web

HTTP BASIC

- Il client tenta di accedere a una risorsa protetta.
- Il server risponde con 401 Unauthorized e un header WWW-Authenticate: Basic realm="Protected".
- Il client reinvia la richiesta includendo l'header Authorization: Basic <dati_base64>.
- Il server verifica le credenziali e, se valide, concede l'accesso.

Vantaggi:

- Facile da implementare.
- Supportato da tutti i browser

Svantaggi:

- Invia le credenziali in ogni richiesta (rischioso senza HTTPS)
- La codifica Base64 non è crittografia → un attaccante può decodificarla facilmente
- Nessuna protezione contro attacchi replay

Autenticazione HTTP Digest

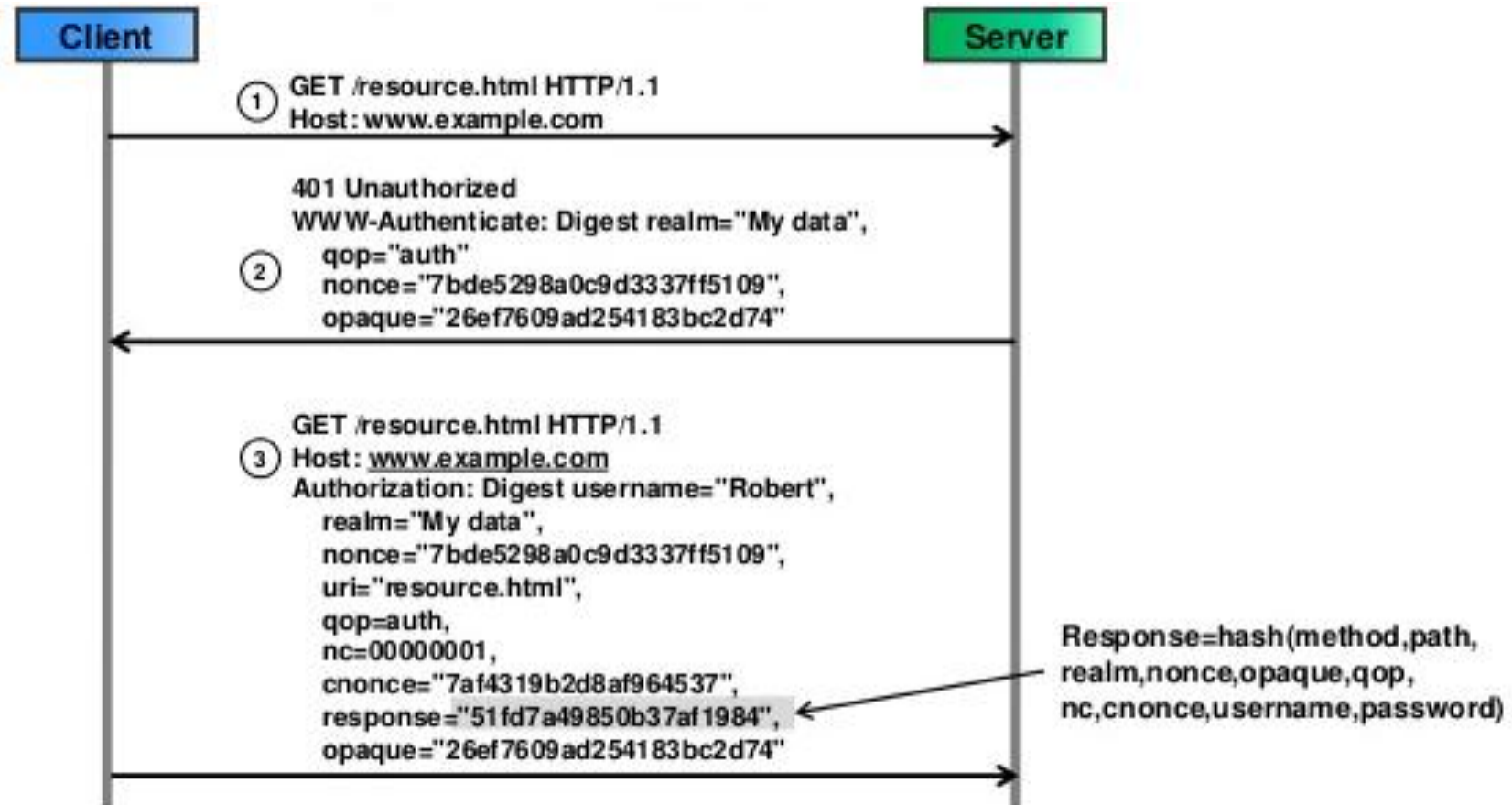
- Il client richiede la risorsa
- Il server risponde negando l'autorizzazione (401) e invia un challenge che contiene un realm e un **nonce** generato casualmente
- Il client calcola l'hash MD5 di una stringa che contiene username, realm, password, nonce del server, nonce generato dal client e altre informazioni

Numero pseudo-casuale generato per un unico utilizzo. Dall'espressione inglese «for the nonce» - per l'occasione

Autenticazione HTTP Digest

- Composizione della risposta
 - $HA1 = MD5(\text{username}:\text{realm}:\text{password})$
 - $HA2 = MD5(\text{metodo}:\text{URI risposta})$
 - $\text{risposta} = MD5(HA1:\text{nonce}:\text{nonceCount}:\text{clientNonce}:\text{qop}:HA2)$
 - **qop** (*quality of protection*) tipicamente “auth” definisce il livello di protezione e quindi di cifratura, eventualmente multipla
- La password in chiaro *non viaggia mai*

Autenticazione HTTP Digest



HTTP DIGEST

- Il client tenta di accedere a una risorsa protetta.
- Il server risponde con 401 Unauthorized e un nonce (un valore casuale univoco per prevenire attacchi replay).
- Il client calcola un hash MD5 combinando username, password, il nonce e l'URI della richiesta.
- Il server verifica l'hash senza ricevere la password in chiaro.

Vantaggi:

- La password non viene mai inviata in chiaro.
- Protezione contro attacchi replay grazie al nonce

Svantaggi:

Maggiore complessità rispetto a Basic Auth

Più lento per via del calcolo degli hash

Limitazioni di HTTP

- Protocollo *stateless*
 - Non c'è controllo del tracciamento dell'attività del client, cioè la **sessione**
- Non ci sono meccanismi interni di sicurezza

Gestione della sessione

- URL rewriting
 - Inserimento dei dati utili a tracciare l'attività dell'utente già nel corpo della URL, ad esempio nomi appropriati delle cartelle nel percorso
- Esempio:
 - http://www.pets.com/show_a_product.php?product_id=7
 - <http://www.pets.com/parrots/norwegian-blue/>
- Campi nascosti nelle form
 - Contengono le informazioni di tracciamento
- Cookie
- Sessioni SSL

Gestione della sessione esempi

1.Cookie di sessione

1. Vengono salvati nel browser e inviati con ogni richiesta HTTP.
2. Scadono automaticamente quando il browser viene chiuso.
3. Utilizzati per mantenere autenticazione e preferenze utente.

2.Local Storage e Session Storage

1. **Session Storage**: Memorizza dati per la durata della sessione (si cancella alla chiusura del browser).
2. **Local Storage**: Memorizza dati in modo persistente fino alla cancellazione manuale.
3. Utilizzati per salvare dati lato client senza inviarli al server.

3.Token di autenticazione (JWT, OAuth)

1. Utilizzati in applicazioni moderne per gestire sessioni senza stato (stateless).
2. Salvati in cookie sicuri o Local Storage.

4.Sessioni lato server

1. Il server memorizza lo stato della sessione (es. in database o memoria cache).
2. Il client invia un identificatore univoco della sessione (es. ID in un cookie).